PRESENTATION & SOURCE CODE GUIDE

ขั้นตอนการนำเสนอระบบ & Source Code ต่อคณะกรรมการ

ระบบจองห้องประชุมออนไลน์ โรงเรียนพรหมคีรีพิทยาคม (My Booking System)

Framework: Next.js 16 (App Router) + React 19 **Database & Realtime:** Supabase (PostgreSQL + RLS)

Security: JWT (HMAC SHA-256) + HTTP-Only Cookie + Rate Limit

★ ตารางสรุปลำดับการเปิดไฟล์โชว์กรรมการ (Quick Checklist)

ลำดับ	หัวข้อที่นำเสนอ	ไฟล์ที่ต้องเปิดใน VS Code	จุดเด่นที่ชี้ให้กรรมการดู
ช่วงที่ 1 (1-2 นาที)	ภาพรวมสถาปัตยกรรม & Tech Stack	`package.json` โครงสร้างโฟลเดอร์ `app/`, `lib/`	ใช้ Next.js 16, React 19, Supabase, Web Push, Nodemailer, XLSX
ช่วงที่ 2 (2-3 นาที)	หน้าแรก Dashboard & การซ่อนข้อมูลส่วนบุคคล (PDPA)	`app/page.tsx` `app/api/bookings/route.ts` (บรรทัด 54–70)	ถ้าไม่ใช่ Admin จะไม่ดึงคอลัมน์ `details` ป้องกันเบอร์โทรหลุดไปหน้าเว็บ
ช่วงที่ 3 (3-4 นาที)	ฟอร์มการจอง & ลอจิกเช็คเวลาชนกัน (Overlap Check)	`components/BookingForm.tsx` `app/api/bookings/route.ts` (บรรทัด 89–199)	จองได้ทั้งวันเดียว/หลายวัน, Rate Limit 5 ครั้ง/นาที, เช็คเวลาซ้อนทับ, ป้องกัน XSS
ช่วงที่ 4 (3-4 นาที)	ระบบความปลอดภัย & การยืนยันตัวตน Admin	`app/api/admin/login/route.ts` `lib/jwt.ts`, `proxy.ts` `supabase/security.sql`	SHA-256 Password Hash, JWT ลง HTTP-Only Cookie, Proxy กั้นหน้า Admin, RLS บน DB
ช่วงที่ 5 (2-3 นาที)	การอนุมัติ, แจ้งเตือน Real-time & ออกรายงาน Excel	`app/admin/page.tsx` `app/api/admin/update/route.ts` `components/BookingStatusWatcher.tsx` `components/ExportModal.tsx`	อนุมัติแล้วยิง Web Push + Realtime Popup ทันที, ส่งออก Excel/พิมพ์สรุปราชการ

1 รายละเอียดการนำเสนอทีละขั้นตอน (เปิดหน้าเว็บคู่กับโค้ด)

ขั้นตอนที่ 1: แนะนำภาพรวมและโครงสร้างโปรเจกต์ (Architecture & Tech Stack)

ใช้เวลา 1–2 นาที

📂 **ไฟล์ที่เปิดโชว์:**

- `package.json` (บรรทัด 11–24) — โชว์ Library หลักที่ใช้ในระบบ
- แถบ Explorer ด้านซ้าย — ชี้ให้เห็นการแยกโครงสร้าง `app/` (หน้าเว็บและ API Routes), `components/` (UI แยกส่วน), และ `lib/` (ระบบรักษาความปลอดภัยและตัวเชื่อมฐานข้อมูล)

🗣️ **บทพูดแนะนำ:**

"ระบบจองห้องประชุมออนไลน์ของโรงเรียนพรหมคีรีพิทยาคม พัฒนาในรูปแบบ Full-stack Web Application ด้วย Next.js 16 (App Router) และ React 19 เชื่อมต่อฐานข้อมูล PostgreSQL บน Supabase โดยออกแบบแยกส่วน Frontend Component กับ Backend API Routes ออกจากกันอย่างชัดเจนเพื่อง่ายต่อการดูแลรักษาและปลอดภัยครับ"

ขั้นตอนที่ 2: หน้าแรก (Public Dashboard) & การปกป้องข้อมูลส่วนบุคคล (PDPA)

ใช้เวลา 2–3 นาที

📂 **ไฟล์ที่เปิดโชว์:**

- `app/page.tsx` (บรรทัด 24–60) — การดึงข้อมูลตารางจองมาแสดงใน `DashboardStats`, `RoomUsageStats` และปฏิทิน `Calendar`
- `app/api/bookings/route.ts` (บรรทัด 38–70 ในฟังก์ชัน `GET`) — **จุดขายสำคัญ!** โชว์การเช็คสิทธิ์ว่าถ้าเป็นผู้ใช้ทั่วไป ระบบจะตัดคอลัมน์ `details` ออกตั้งแต่ระดับ SQL SELECT

🗣️ **บทพูดแนะนำ:**

"เมื่อผู้ใช้เข้ามาหน้าแรก จะเห็นสถิติและปฏิทินการใช้ห้องประชุมทั้ง 3 ห้องได้ทันทีโดยไม่ต้องล็อกอิน แต่จุดที่เราให้ความสำคัญมากคือเรื่อง PDPA ครับ ในไฟล์ `app/api/bookings/route.ts` บรรทัดที่ 54 ถ้าผู้เรียก API ไม่ใช่ Admin ที่มี JWT Cookie ระบบจะไม่ดึงคอลัมน์ `details` ที่เก็บเบอร์โทรศัพท์และหมายเหตุออกมาเลย ทำให้ต่อให้เปิด Inspector ดู Network ข้อมูลส่วนบุคคลก็จะไม่รั่วไหลครับ"

ขั้นตอนที่ 3: ระบบจองห้องประชุม & การตรวจสอบเวลาชนกัน (Booking & Overlap Logic)

ใช้เวลา 3–4 นาที

📂 **ไฟล์ที่เปิดโชว์:**

- `components/BookingForm.tsx` (บรรทัด 29–118) — โชว์การรองรับโหมดจองวันเดียว (`single`) และหลายวัน (`multiple`) พร้อมการขอสิทธิ์ Web Push Notification (`subscribeToPush`)
- `app/api/bookings/route.ts` (บรรทัด 89–199 ในฟังก์ชัน `POST`) — ชี้ให้ดู 4 ด่านป้องกันหลังบ้าน:
  1. **บรรทัด 91–99 (Rate Limit):** จำกัดการจองไม่เกิน 5 ครั้ง/นาที ต่อ IP ป้องกันบอทสแปม
  2. **บรรทัด 124–133 (Date/Time Validation):** ห้ามเวลาสิ้นสุดน้อยกว่าเวลาเริ่ม และห้ามจองย้อนหลัง
  3. **บรรทัด 135–152 (Overlap Check):** ตรวจสอบว่ามีรายการจองห้องเดียวกันในวันและเวลาที่ทับซ้อนกันหรือไม่
  4. **บรรทัด 154–159 (XSS Sanitization):** แปลงเครื่องหมาย `<` และ `>` ป้องกันการฝังสคริปต์อันตราย

🗣️ **บทพูดแนะนำ:**

"หน้าฟอร์มจองออกแบบให้ครูสามารถเลือกจองแบบวันเดียวหรือหลายวันพร้อมกันได้ เมื่อกดส่ง ข้อมูลจะวิ่งเข้า `POST /api/bookings` ซึ่งเราใช้หลัก Zero Trust คือตรวจสอบซ้ำที่ Server ทุกครั้ง เริ่มจาก Rate Limit กันสแปม, เช็ควันเวลาห้ามย้อนหลัง และหัวใจสำคัญคือบรรทัด 136 การเช็คเวลาชนกัน (Overlap Check) ด้วยสมการ `start_time < end_time` และ `end_time > start_time` ถ้าไม่มีคิวชนกันจึงจะบันทึกเป็นสถานะ 'รออนุมัติ' แล้วส่งทั้ง Email และ Web Push ไปแจ้งเตือนแอดมินทันทีครับ"

ขั้นตอนที่ 4: ระบบความปลอดภัยและการยืนยันตัวตนผู้ดูแลระบบ (Admin Auth & Security)

ใช้เวลา 3–4 นาที

📂 **ไฟล์ที่เปิดโชว์:**

- `app/api/admin/login/route.ts` (บรรทัด 37–62) — การนำรหัสผ่านไป Hash ด้วย **SHA-256** เทียบกับค่าเป้าหมาย แล้วออก JWT Token เก็บใน **HTTP-Only Cookie** (`httpOnly: true` , `sameSite: 'strict'`)
- `lib/jwt.ts` (บรรทัด 51–101) — โชว์ฟังก์ชัน `signJWT` และ `verifyJWT` ที่เขียนด้วย Web Crypto API มาตรฐาน (HMAC SHA-256)
- `proxy.ts` (บรรทัด 5–31) — ตัวดักจับ Request ของ Next.js 16 ที่ตรวจ JWT Cookie ทุกครั้งก่อนอนุญาตให้เข้าหน้า `/admin/*`
- `supabase/security.sql` (บรรทัด 1–33) — กฎ **Row Level Security (RLS)** บนฐานข้อมูลที่บล็อกการแก้ไข/ลบจากฝั่ง Client โดยตรง

🗣️ **บทพูดแนะนำ:**

"ด้านความปลอดภัยของแอดมิน เราป้องกันไว้ 3 ชั้นครับ ชั้นแรกตอนล็อกอิน รหัสผ่านจะถูกแปลงเป็น SHA-256 และเมื่อถูกต้อง ระบบจะสร้าง JWT Token ฝังลงใน HTTP-Only Cookie ซึ่งปลอดภัยกว่าการเก็บใน localStorage เพราะสคริปต์ฝั่งหน้าเว็บไม่สามารถอ่านหรือขโมยไปได้ ชั้นที่สองเรามี `proxy.ts` คอยดักตรวจ Cookie ทุกหน้าของ `/admin` และชั้นที่สามบนฐานข้อมูล Supabase เราเปิด Row Level Security (RLS) บังคับให้การแก้ไขหรือลบข้อมูลต้องทำผ่าน `supabaseAdmin` (Service Role) บน Server ที่ตรวจ JWT ผ่านแล้วเท่านั้นครับ"

ขั้นตอนที่ 5: ระบบอนุมัติ, การแจ้งเตือน Real-time & การออกรายงานราชการ

ใช้เวลา 2–3 นาที

📂 **ไฟล์ที่เปิดโชว์:**

- `app/admin/page.tsx` คู่กับ `app/api/admin/update/route.ts` (บรรทัด 61–132) — เมื่อแอดมินกดอนุมัติ/ยกเลิก ระบบจะอัปเดตฐานข้อมูล ยิง Web Push แจ้งผู้จอง และซิงค์ข้อมูลเข้า Google Sheets เบื้องหลัง
- `components/BookingStatusWatcher.tsx` (บรรทัด 1–90) — การใช้ **Supabase Realtime** ดักฟัง Event `UPDATE` เพื่อเด้งป๊อปอัปแจ้งผลผู้จองบนหน้าเว็บแบบวินาทีต่อวินาที
- `components/ExportModal.tsx` (บรรทัด 38–100) — ฟีเจอร์กรองข้อมูลตามเดือน/ปี/ห้องประชุม เพื่อ Export เป็นไฟล์ Excel (`.xlsx`) หรือสั่งพิมพ์สรุปรายงานราชการ

🗣️ **บทพูดแนะนำ:**

"เมื่อแอดมินกดอนุมัติหรือยกเลิกพร้อมระบุเหตุผล ระบบหลังบ้านใน `/api/admin/update` จะส่ง Web Push Notification กลับไปยังมือถือหรือคอมพิวเตอร์ของผู้จองทันที และถ้าผู้จองยังเปิดหน้าเว็บอยู่ คอมโพเนนต์ `BookingStatusWatcher.tsx` ที่เชื่อมกับ Supabase Realtime ก็จะเด้งป๊อปอัปแจ้งผลทันทีโดยไม่ต้องรีเฟรชหน้าเว็บ นอกจากนี้แอดมินยังสามารถกดปุ่มออกรายงานผ่าน `ExportModal.tsx` เพื่อดาวน์โหลดเป็นไฟล์ Excel หรือพิมพ์สรุปสถิติการใช้ห้องประชุมรายเดือนเสนอผู้บริหารได้ทันทีครับ"

2 สิ่งที่คุณต้องรู้เกี่ยวกับระบบนี้ & เก็งคำถามกรรมการ (Q&A)

? คำถามที่ 1: ระบบเช็คยังไงว่าเวลาจองชนกันหรือไม่? ถ้าคนแรกจอง 09:00–11:00 คนที่สองจอง 10:00–12:00 ระบบรู้ได้อย่างไร?

คำตอบ: ในไฟล์ `app/api/bookings/route.ts` (บรรทัด 136–143) เราใช้ทฤษฎีการทับซ้อนของช่วงเวลา (Interval Overlap) โดย Query หาการจองในห้องเดียวกันและวันเดียวกัน (ที่ไม่ได้ถูกยกเลิก) ด้วยเงื่อนไขคู่:
`.lt('start_time', end_time)` (เวลาเริ่มเดิม < เวลาจบใหม่) **และ** `.gt('end_time', start_time)` (เวลาจบเดิม > เวลาเริ่มใหม่)
สมการนี้ดักจับการเหลื่อมเวลาได้ครบทุกกรณี (ทับหัว ทับท้าย คร่อมกลาง หรือเวลาตรงกันเป๊ะ) แต่ถ้าจองต่อกันพอดี เช่น 09:00–10:00 กับ 10:00–11:00 จะไม่ถือว่าชนกันและสามารถจองต่อคิวกันได้ทันทีครับ

? คำถามที่ 2: ทำไมในโฟลเดอร์ `lib/` ถึงมีทั้ง `supabase.ts` และ `supabase-admin.ts` ต่างกันอย่างไร?

คำตอบ: แยกตามระดับสิทธิ์ความปลอดภัย (Principle of Least Privilege) ครับ:
• `lib/supabase.ts` ใช้ Public Anon Key สำหรับการดึงตารางจองและส่งคำขอจองใหม่ ซึ่งถูกจำกัดสิทธิ์ด้วย **Row Level Security (RLS)** ใน `supabase/security.sql` ให้ทำได้แค่ `SELECT` กับ `INSERT` เท่านั้น
• `lib/supabase-admin.ts` ใช้ Service Role Key ซึ่งทำงานเฉพาะฝั่ง Server ใน API ของ Admin (หลังจากตรวจสอบ JWT ผ่านแล้ว) เพื่อใช้สิทธิ์ระดับผู้ดูแลในการ `UPDATE` สถานะหรือ `DELETE` รายการจอง ป้องกันผู้ไม่หวังดียิงแก้ไขฐานข้อมูลจากหน้าบ้านโดยตรงครับ

? คำถามที่ 3: ผู้ใช้ทั่วไปไม่ต้องสมัครสมาชิก (ไม่มีระบบ Login สำหรับผู้จอง) แล้วเขายกเลิกการจองของตัวเองหรือรับแจ้งเตือนได้อย่างไร?

คำตอบ: เราออกแบบเพื่อลดขั้นตอนให้ครูและบุคลากรใช้งานได้สะดวกที่สุดโดยไม่ต้องจำรหัสผ่านครับ:
1. **การขอยกเลิกการจอง:** ในไฟล์ `app/api/bookings/request-change/route.ts` ระบบจะให้ผู้จองกรอก **เบอร์โทรศัพท์ที่ใช้ตอนจอง** เพื่อยืนยันตัวตน หากตรงกับข้อมูลในระบบจึงจะอนุญาตให้ยกเลิกได้
2. **การแจ้งเตือน:** ตอนกดจอง ระบบจะบันทึก `booking.id` ไว้ใน `sessionStorage` ของเบราว์เซอร์เครื่องนั้นและผูก **Web Push Subscription** ไว้กับรายการจอง ทำให้ส่งแจ้งเตือนกลับไปถูกเครื่องโดยไม่ต้องมีบัญชีผู้ใช้ครับ

? คำถามที่ 4: ข้อมูลฝ่ายงาน/กลุ่มสาระ เบอร์โทรศัพท์ และ Push Subscription เก็บไว้ที่ไหนในฐานข้อมูล?

คำตอบ: เพื่อความยืดหยุ่นโดยไม่ต้องแก้ไขโครงสร้างตารางหลักหลายคอลัมน์ ระบบนำข้อมูลหมายเหตุ, กลุ่มสาระ (`---DEPARTMENT---`), เบอร์ติดต่อ (`---CONTACT_INFO---`), และรหัสรับแจ้งเตือน (`---PUSH_SUB---`) มาคั่นด้วย Delimiter เก็บรวมไว้ในคอลัมน์ `details` (ดูที่ `app/api/bookings/route.ts` บรรทัด 161–179) และเมื่อ Admin เปิดดูหรือกด Export Excel ระบบจะมีฟังก์ชัน `parseBooking` แยกข้อมูลแต่ละส่วนออกมาแสดงผลและคำนวณสถิติแยกตามกลุ่มสาระได้อย่างแม่นยำครับ

? คำถามที่ 5: ไฟล์ `proxy.ts` และ `app/api/cron/keepalive/route.ts` มีไว้ทำอะไร?

คำตอบ:
• `proxy.ts` คือมาตรฐานใหม่ของระบบ Middleware ใน **Next.js 16** ที่คอยดักทุก Request ที่วิ่งเข้า `/admin/*` เพื่อตรวจความถูกต้องของ JWT Cookie ก่อนเรนเดอร์หน้าเว็บ
• `app/api/cron/keepalive/route.ts` ทำงานร่วมกับ `vercel.json` (ตั้งเวลา Cron Job ทุกวันจันทร์และพฤหัสบดี) เพื่อยิง Query เบาๆ ไปปลุกฐานข้อมูล Supabase ป้องกันไม่ให้โปรเจกต์บน Supabase Free Tier ถูกระงับชั่วคราว (Auto-pause) เมื่อไม่มีการใช้งานช่วงปิดเทอมครับ

? คำถามที่ 6: ถ้ามีคนพยายามสุ่มรหัสผ่าน Admin (Brute-force) หรือกดส่งฟอร์มรัวๆ ระบบรับมืออย่างไร?

คำตอบ: เรามีโมดูล `lib/rate-limit.ts` ที่ดึง IP Address ของผู้เรียกผ่าน Header `x-forwarded-for` มานับจำนวนครั้งต่อ 1 นาทีแยกตามความเสี่ยงของแต่ละ API เช่น:

• ส่งรายงานปัญหา (`/api/feedback`): จำกัด **3 ครั้ง/นาที** | จองห้องประชุม/ขอยกเลิก (`POST /api/bookings`): จำกัด **5 ครั้ง/นาที**

• ล็อกอิน Admin (`/api/admin/login`): จำกัด **10 ครั้ง/นาที** (หากเกินจะคืนค่า HTTP `429 Too Many Requests` ทันที)